BCSE408L · CLOUD COMPUTING · FINAL PROJECT REPORT · ACADEMIC YEAR 2026–2027 · VIT CHENNAI

Women360

A cloud-native women's health and wellness SaaS platform

Project Title	Women360: a cloud-native women's health and wellness SaaS platform
Team Member 1: Name & Reg. No.	Pavithra Uthrah R K, 23BCE1818
Team Member 2: Name & Reg. No.	Asmi Singh, [REG. NO. - TO FILL]
Faculty / Slot	[FACULTY / SLOT - TO FILL]
Public Live URL	https://women-360.vercel.app (API: https://women360-api.onrender.com)
GitHub Repository URL	https://github.com/uthrahh/Women360
Submission Deadline	28/10/2026

> **Important:** This report is a supporting document for project evaluation. Project marks are based on the reviews and final demonstration completed during the semester, and not on this report submission alone.

1. Abstract

Women's health tools are usually single-purpose: one app for periods, another for food, another for sleep or exercise. None of them looks across the data a person already records, most assume one fixed cycle length, and almost all are designed for a young, confident phone user. Women360 is a cloud-native SaaS platform that brings menstrual health, nutrition, activity, sleep, mental wellbeing, physical health measurements, medicines, goals and habits into one responsive web application for women from 18 to 100 and beyond.

Its primary goal is **preventive care**: it turns what a woman logs into early, non-diagnostic prompts, including a screening schedule based on India's national programme and WHO guidance, and observations drawn from her own baseline rather than from population averages. Its second requirement is **ease of use**: one-tap logging, templates and sensible defaults, and a Senior Mode that replaces the dashboard with a handful of large, plain-language tiles.

The system has three roles (Woman, Wellness Coach, Admin) enforced on the server. It is a React 18 + TypeScript single-page application hosted on Vercel, talking over HTTPS to a Node.js/Express REST API (about 110 endpoints) hosted on Render, which stores data in a managed PostgreSQL database on Supabase through the Prisma ORM. Source control, automated checks and deployment are driven from GitHub. The application is live at the URL above.

2. Introduction

Women360 started as the BCSE408L Cloud Computing project and has been built as a real product rather than a coursework demonstration. The Software Requirements Specification (SRS) fixes the scope: secure registration and login, role-based access, profile management, health and wellness tracking with full

CRUD, dashboards and reports, notifications, a cloud database, REST APIs, and public deployment on a cloud platform. The project must demonstrate cloud concepts, and is not considered complete if it runs only on localhost.

The product is deliberately a *wellness, tracking and decision-support* application. It does not diagnose, prescribe, or present a screening schedule as a clinical instruction (SRS §12).

3. Problem Statement

- **Fragmentation.** A woman's cycle, sleep, food, movement and mood affect one another, but each is tracked in a separate place, so nobody can ask "why is my period late?" or "why has my mood been low?" against her own data.
- **False precision.** Fixed assumptions (a 28-day cycle, ovulation on day 14) are wrong for many people and unsafe when read as a contraception signal, especially with irregular cycles, thyroid conditions, PCOS or perimenopause.
- **Tracking fatigue.** Retyping the same meal or activity every day is the most common reason people stop. Shame-based feedback (red numbers, lost streaks) can also encourage unhealthy behaviour.
- **Prevention is invisible.** Most tools record what has happened; few remind a woman which check-ups are due at her age, or in her country.
- **Age and accessibility.** Interfaces are designed for the young. A woman in her seventies needs a different structure, not just a bigger font.
- **Privacy.** Health data is sensitive; users should be able to see, export and erase it, and share it only deliberately.

4. Existing System

Existing consumer apps fall into separate categories: cycle trackers, calorie counters, sleep trackers, fitness apps, meditation apps and medication reminders. Each treats its own metric in isolation and has a recognised weakness:

Approach	Limitation addressed by Women360
Cycle trackers	Single-date predictions from a fixed formula; weak for irregular cycles and perimenopause; fertile-window estimates easily misread as contraception.
Calorie / nutrition apps	Every entry needs a full form; red/green feedback and streaks; no link to how the person feels or sleeps.
Sleep apps	A single score can make people anxious about a number and contradict how they actually feel.
Medication reminders	Reminders without any view of adherence over time or its link to wellbeing.
Generic health dashboards	Not designed for older users; no preventive-care schedule; limited control over data export or deletion.

5. Proposed System

Women360 is a single cloud-hosted application with a stateless REST API and a managed database. Its main advantages:

- **One place for the whole picture,** with a cross-domain "explain this" engine that compares recent sleep, food, activity, hydration, stress and weight against *that person's own* baseline and reports ranked, disclaimed observations. A late period is never simply labelled "maybe pregnant", and low mood is never simply "PMS".
- **Honest cycle predictions:** median and spread over her own cycles, shown as a range with a confidence level, with a persistent "not a birth control method" notice.
- **Preventive screening:** a per-age schedule (blood pressure, blood sugar, oral, breast and cervical screening) with named sources, "next due" dates and one-tap recording.
- **Low-effort logging:** one-tap quick logs, "log again" meal chips, one-tap habit ticks, forms that never pre-fill an unentered value.
- **Senior Mode:** a structural change to navigation and information architecture, not a zoom.
- **Privacy by design:** every record belongs to one user, data can be exported and the account erased, and coach access is role-limited.
- **Cloud-native delivery:** free-tier managed services, git-driven deployment, database migrations applied automatically on deploy, and a CI pipeline.

6. Requirement Analysis

6.1 Users

Role	Needs (from SRS §2.3 and §7)
Woman	Register and log in; manage her profile; create, view, edit and delete her own records; see dashboards, trends and reports; manage goals; receive reminders; control coach access.
Wellness Coach	Authenticate; see only the information a woman has authorised; add coaching notes.
Admin	Manage users and roles; manage educational content; review audit logs and feedback; configure the system.

6.2 Summary

The SRS defines 24 functional requirements (FR-01 to FR-24), 10 non-functional requirements (NFR-01 to NFR-10), 10 UI requirements (UI-01 to UI-10), 6 business rules and 10 acceptance criteria. Sections 7 and 8 report, requirement by requirement, what has been implemented and what has not.

7. Functional Requirements

Status key: **Implemented** · **Partial** · **Not in the web UI**.

ID	Requirement	Status	What exists
FR-01	Registration	**Implemented**	Name, email, date of birth and password; password policy (12+ characters, not common, no sequences, not derived from name or email) checked on client and server.
FR-02	Login	**Implemented**	JWT access and refresh tokens, silent refresh on expiry, database-backed lockout, "sign out everywhere", optional TOTP two-factor.
FR-03	Role-based access	**Partial**	Woman, Coach and Admin enforced by server middleware and covered by tests. The web UI exposes the Woman role only.
FR-04	Profile management	**Implemented**	Settings: profile, date of birth, life stage, emergency contact, accessibility and Senior Mode essentials.
FR-05	Health history	**Partial**	Allergies and current medicines captured at onboarding and stored; conditions and family history exist in the data model and API.
FR-06	Menstrual tracking	**Implemented**	Period/flow, symptoms, pain, energy, mood, context tags; calendar, trends and range prediction with confidence.
FR-07	Reproductive health	**Partial**	Life stage (reproductive, perimenopause, menopause, postmenopause) shapes predictions and prompts. Contraception and pregnancy records are not implemented.
FR-08	Nutrition tracking	**Implemented**	Meals with macros (one decimal), hydration, fruit and vegetables, goals, "log again" chips.
FR-09	Physical activity	**Implemented**	Entries with type, duration and intensity; editable one-tap quick log; weekly chart. Step counts are stored; automatic sync from a watch is not built.
FR-10	Sleep tracking	**Implemented**	Duration, bedtime, wake time, 1–5 self-rating, bedtime consistency.
FR-11	Mental wellbeing	**Implemented**	Mood, stress and energy; guided breathing exercise. Social-connection indicators are not captured.
FR-12	Physical health	**Implemented**	Weight, blood pressure, resting heart rate; medicines with dose times and adherence; appointments.
FR-13	Preventive care	**Implemented**	Screening schedule from India (NPCDCS) and WHO sources, next-due dates, one-tap records, appointments, in-app reminders. Marked as not yet reviewed by a clinician.
FR-14	Goals	**Implemented**	Three goal kinds (reach a number, build a total, do one thing) plus separate daily habits with streaks.
FR-15	Dashboard	**Implemented**	Cycle card, this-week rings, upcoming appointments and a to-do list built from the person's own data.

FR-16	Trend analysis	**Implemented**	Insights page, cross-metric correlations over 90 days (only strong ones shown), baseline-deviation explainers.
FR-17	Search and filtering	**Partial**	Date-range filters, report periods, Learn library browsing. There is no global search.
FR-18	Notifications	**Partial**	In-app notification centre and in-app reminders. No push or email delivery.
FR-19	Reports	**Implemented**	7/30/90-day health report generated from stored data and exported as a PDF.
FR-20	Coach module	**API only**	Assignment, sharing consent, limited summary and coach notes are implemented and tested on the server. There is no coach screen in the web UI.
FR-21	Administration	**API only**	User list, role change, deletion, audit log, feedback review and educational-content management exist as protected endpoints, with optional IP allow-list and required two-factor. No admin screen. Admin accounts are created only by a command-line script.
FR-22	CRUD	**Implemented**	Create, read, update and delete across the tracking modules, each with confirmation on delete.
FR-23	Cloud database	**Implemented**	Supabase-hosted PostgreSQL.
FR-24	REST APIs	**Implemented**	About 110 endpoints under `/api/v1` across 19 modules, used by the application.

8. Non-Functional Requirements

Requirement	How it is addressed
Security (NFR-01)	Passwords hashed with Argon2id. Short-lived access tokens (15 minutes) and refresh tokens (30 days). Helmet security headers, CORS allow-list, rate limiting (20 requests per 15 minutes on the authentication endpoints, 120 per minute on the API). Every request body validated with zod. Sign-in lockout (5 per account, 40 per IP, 15 minutes) stored in the database so it survives restarts. Optional TOTP two-factor with AES-256-GCM-encrypted secrets and backup codes. Admin accounts cannot be created through the API. Security-relevant events are written to an audit log without emails or secrets. The web app sends a Content-Security-Policy and related headers. CI runs a dependency audit.
Privacy (NFR-02)	Each record carries its owner; every service checks ownership before reading or changing it (tested for cross-user access). A person can download everything held about her account and permanently erase it. Coaches see a limited summary and never the health profile or medicines.
Scalability (NFR-03)	Stateless API instances behind a platform load balancer; managed PostgreSQL with indexes on the per-user query paths; static front end served from a CDN. Moving to a paid tier or more instances needs no code change.
Availability (NFR-04)	Managed platforms with a health-check endpoint. On the free tier the API sleeps when idle; the first request after a pause took about 22 seconds in a test on 22 October 2026, and later requests were fast.

Performance (NFR-05)	Route-level code splitting (heavy pages such as Reports are loaded only when visited); data fetched per page; the landing page loads no WebGL.
Reliability (NFR-06)	Schema changes ship as committed migrations applied on deploy; CI fails if the schema and the migrations disagree. Errors return a uniform envelope and failed saves show an error rather than silently dropping data.
Usability (NFR-07)	One-tap logging, templates, progressive disclosure, plain language, no streak guilt, honest empty states, and Senior Mode.
Responsiveness (NFR-08)	Layouts checked at phone and desktop widths with no horizontal scrolling; a bottom navigation bar on phones.
Maintainability (NFR-09)	Strict TypeScript on both sides, ESLint, one folder per domain on the server and a service layer on the client, 305 automated tests, and a written engineering standard in the repository.
Portability (NFR-10)	All configuration through environment variables; a Render blueprint (`render.yaml`) and a step-by-step `DEPLOYMENT.md` reproduce the deployment from the repository.

9. System Architecture

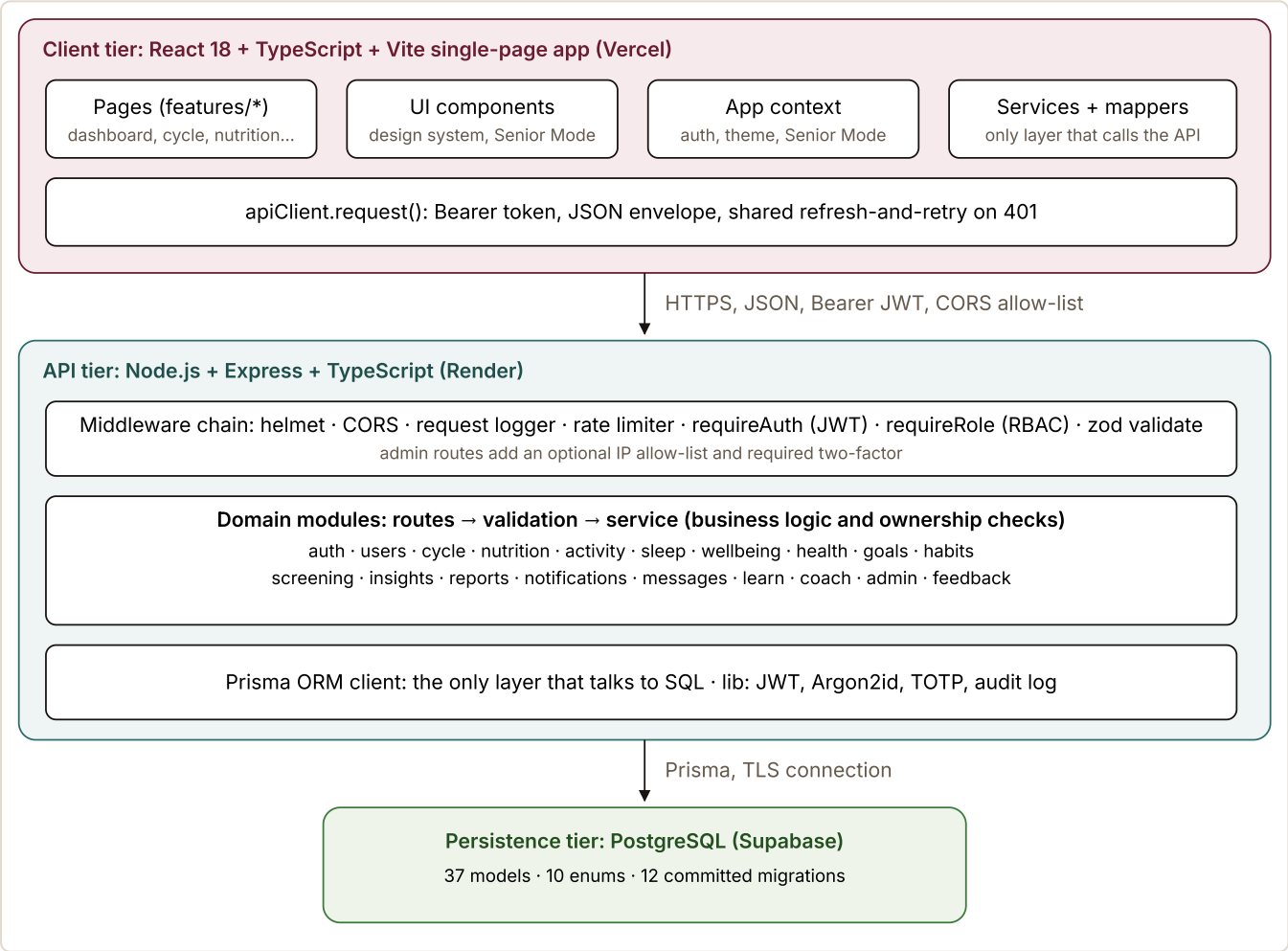


Figure 1. System architecture: three tiers with a single, narrow responsibility per layer.

Request flow. Logging a meal, for example: a page component calls `nutritionService`; `apiClient.request()` adds the Bearer token and sends HTTPS to `/api/v1/nutrition/meals`; Express runs the middleware chain;

the zod schema rejects malformed input; `nutrition.service.ts` applies the business rules and writes through Prisma; the API answers with `{ ok, data }` or `{ ok: false, error }` and the page updates.

Boundaries. No component calls `fetch` directly. A shared `mappers.ts` is the single place that translates the server's `UPPER_SNAKE_CASE` enums and date formats into the client's vocabulary. On the server, routes only wire HTTP to handlers, services hold the logic, and Prisma is the only code that touches SQL.

10. Cloud Architecture

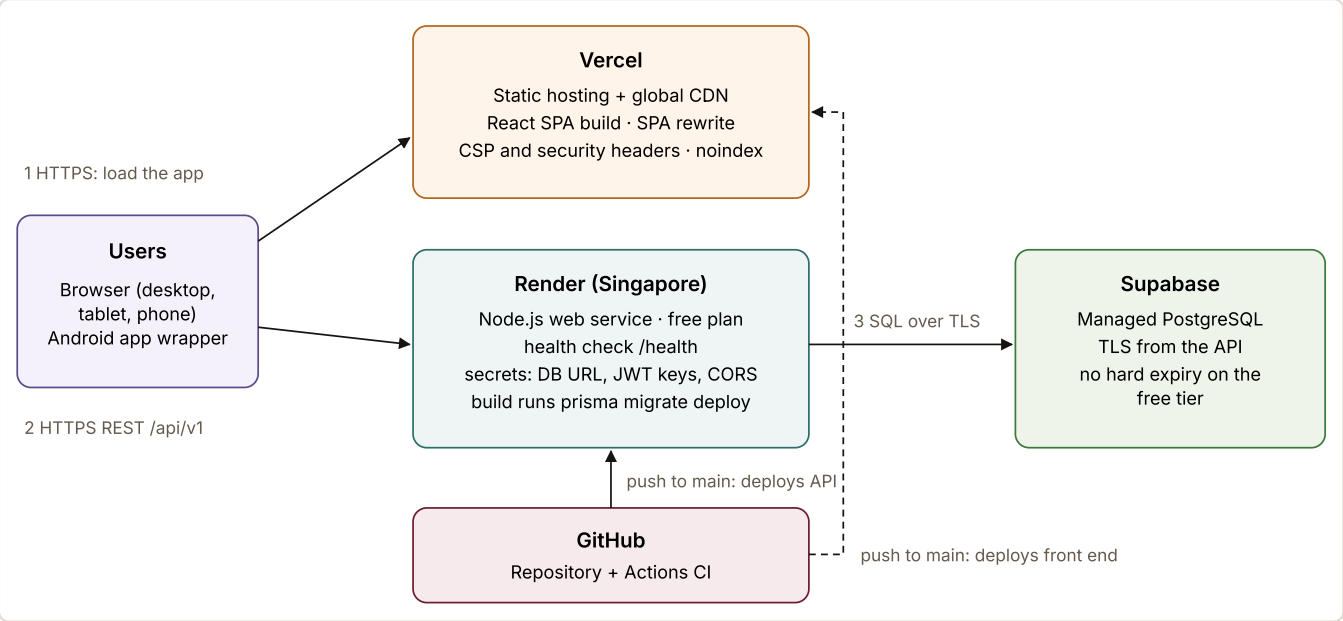


Figure 2. Cloud architecture and deployment topology.

Layer	Service	Role and reason
Front end	Vercel	Hosts the static React build on a CDN, rewrites every route to the SPA, and sends the Content-Security-Policy and other headers (see `vercel.json`). Deploys on every push to `main`.
API	Render	Runs the Express service as a long-lived Node process (not serverless functions). The blueprint `render.yaml` sets the build (`npm ci --include=dev`, Prisma generate, TypeScript build, `prisma migrate deploy`), the start command, the health check and the environment variables.
Database	Supabase	Managed PostgreSQL, an option the SRS approves (§8.2). Chosen because its free tier does not expire after 30 days.
Source and CI	GitHub	Version control and a workflow that runs lint, type-check, tests and build for the server (against a PostgreSQL service container) and lint, type-check, tests and build for the front end.
Mobile (optional)	Capacitor	An Android wrapper that loads the live site, so a phone app and the website always show the same version.

11. Database Design

The data model is a relational PostgreSQL schema of 37 models and 10 enums defined in `server/prisma/schema.prisma` and delivered as 12 committed migrations. Every personal record has a `userId` foreign key with cascade delete, which is what makes "erase my account" a single operation.

Group	Models	Purpose and key relations
Identity and security	User, RefreshToken, PasswordResetToken, TwoFactorBackupCode, AuthThrottle, AuditLog	Account, role (WOMAN, COACH, ADMIN), life stage, session tokens, sign-in lockout counters and an audit trail.
Profile and health	HealthProfile, EmergencyContact, SeniorEssentialPreference, VitalMeasurement, Medication, MedicationLog, Appointment	One-to-one profile data; measurements; medicines with dose times; per-dose taken or skipped log; appointments.
Cycle	CycleProfile, CycleEntry	Her averages and one entry per logged day (period, flow, symptoms, context tags).
Nutrition	NutritionGoal, MealEntry, HydrationLog, FruitVegLog	Daily goals and logged meals, water and servings.
Activity	ActivityGoal, ActivityEntry, ActivityQuickLog, StepsLog	Goals, logged activity, her own one-tap shortcuts, daily steps.
Sleep and mind	SleepEntry, WellbeingEntry	One entry per day each (1–5 quality; 0–4 mood, stress, energy).
Goals and habits	Goal, Habit, HabitLog, GoalCheckIn (retired)	Goals by kind (reach, total, one-off); habits with one log row per day done. Progress and streaks are always computed, never stored.
Prevention	ScreeningRecord	When she last had each screening; the schedule itself lives in code with its sources.
Content and communication	LearnArticle, Notification, Message, Report, Feedback	Educational content, in-app notifications and messages, generated report snapshots, product feedback.
Coaching	CoachAssignment, CoachNote	Consent link between a woman and a coach, and coach notes (optionally visible to her).

Integrity. Required fields and ranges are validated by zod before any write; relations use foreign keys with cascade delete; unique constraints stop duplicate daily rows (one sleep entry, one habit log, one steps log per day). A CI step fails if the schema file differs from what the migrations produce.

12. Module Description

Module	Responsibility
Authentication and users	Register, login, refresh, logout, sign out everywhere, password reset, two-factor, profile, emergency contact, data export and account deletion.

Cycle	Daily entries, calendar of phases, statistics over her own cycles, range prediction with a confidence level, context tags that exclude unusual cycles.
Nutrition	Meals, daily totals computed from the rows, hydration, fruit and vegetables, recent-meal shortcuts.
Activity	Entries, weekly summary, steps, goals, editable quick log.
Sleep and wellbeing	Daily entries, consistency and trend charts, breathing exercise for low mood.
Health	Vitals, medicines with adherence over 7 or 30 days, appointments.
Goals and habits	Goal kinds with computed progress; habits with day or week streaks, amounts and fixable past days.
Screening	Guideline registry (rules, intervals, sources), per-user status and next-due dates, one-tap records.
Insights	Cross-metric correlations and the "explain this" engine for late periods and low mood.
Reports	Snapshots of a chosen period (nutrition, sleep, cycle, activity, wellbeing, goals) and a client-side PDF.
Notifications, messages, learn	In-app notification centre, messages, educational articles (create, update and delete restricted to Admin).
Coach	Women grant and revoke access; coaches read a limited summary and add notes; every view is audited.
Admin	List and manage users and roles, audit log, feedback review, with IP allow-list and two-factor guards.

13. User Interface Design

The interface uses a deep maroon palette on warm neutrals with a serif display face (Fraunces) and Inter for text, defined once as design tokens with a real dark mode. UI primitives (buttons, cards, modals, tabs, toasts, empty, loading and error states) are shared across every page. Figures 3 to 17 are screenshots of the running application; the demonstration data is synthetic.

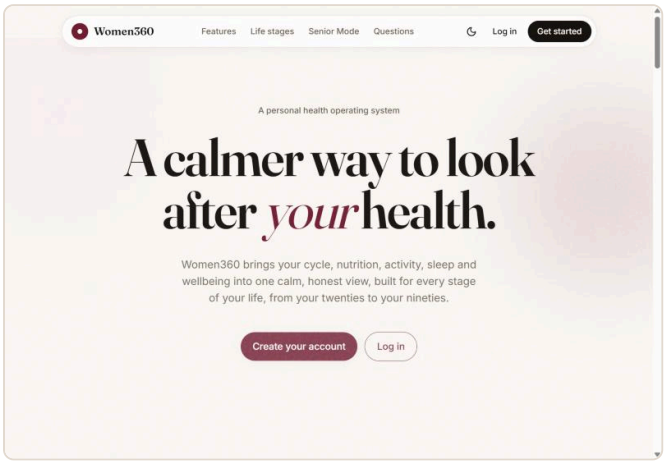


Figure 3. Public landing page (live site).

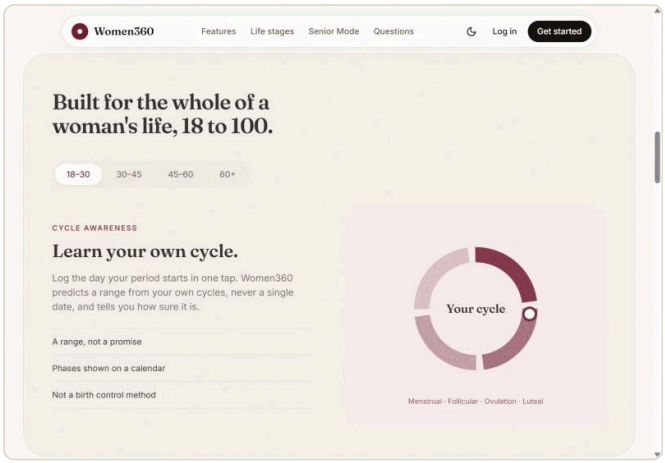


Figure 4. Life-stage tabs, 18 to 100.

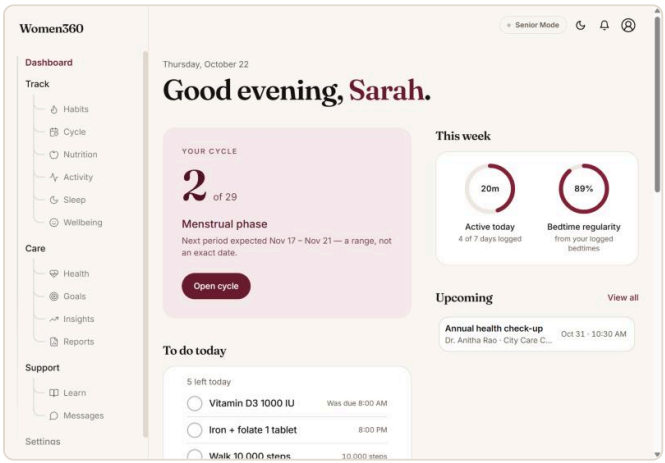


Figure 5. Dashboard.

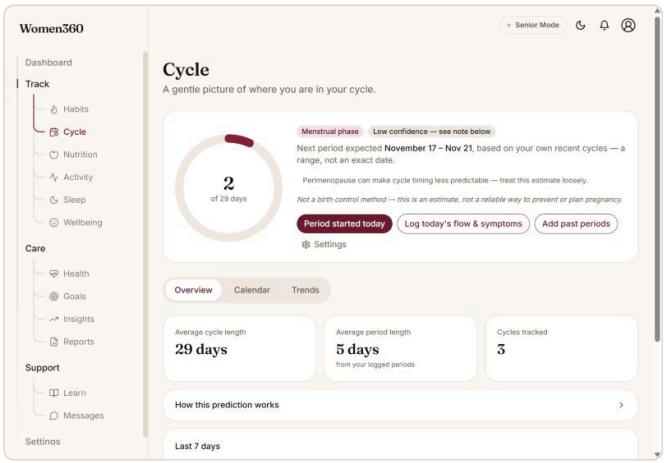


Figure 6. Cycle: a range with a confidence note.

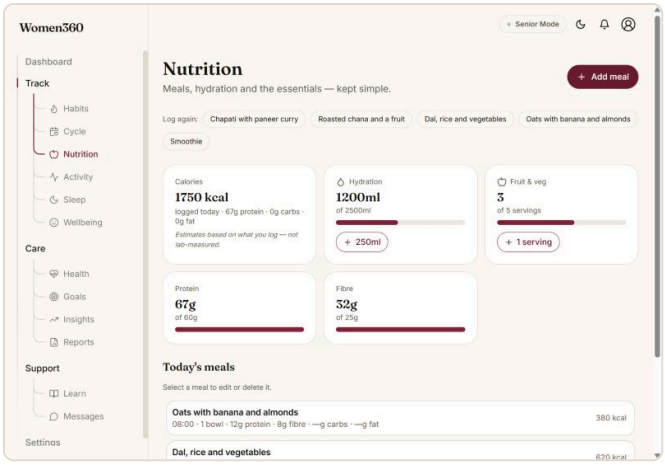


Figure 7. Nutrition with "log again" chips.

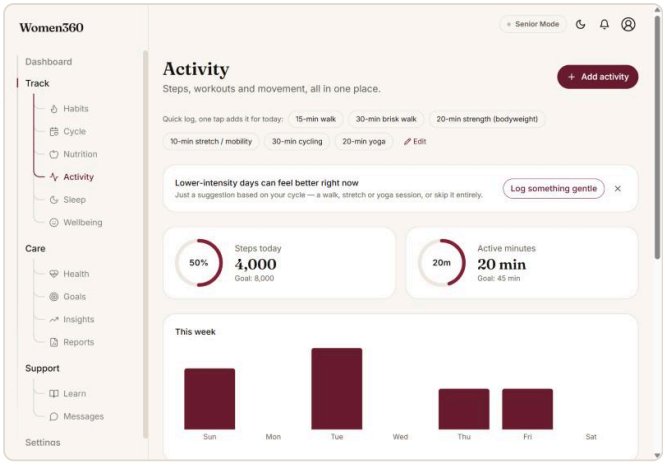


Figure 8. Activity with the one-tap quick log.

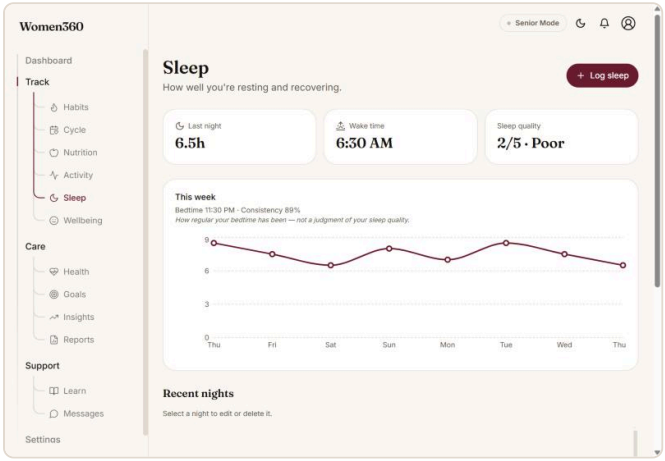


Figure 9. Sleep.

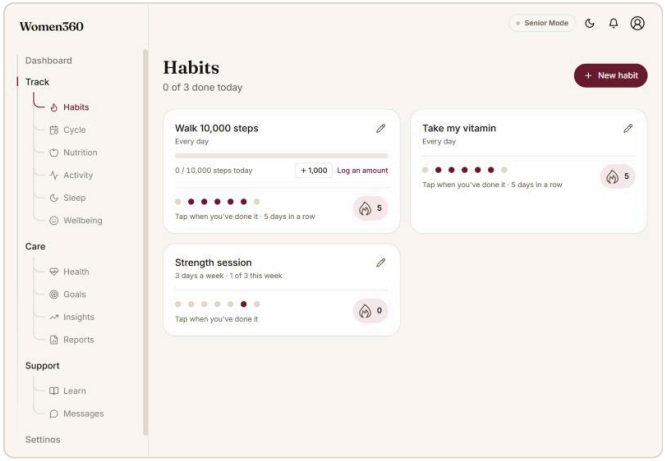


Figure 10. Habits: one-tap ticks and streaks.

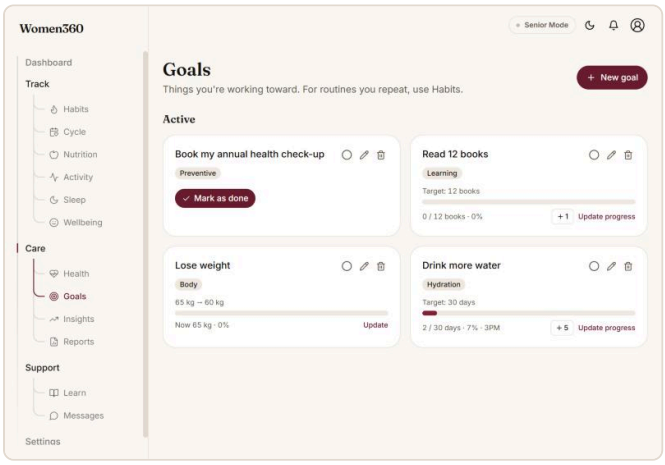


Figure 11. Goals by kind.

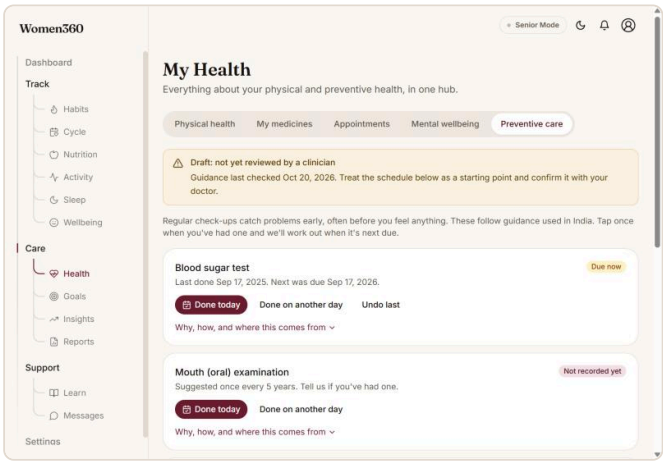


Figure 12. Preventive care schedule.

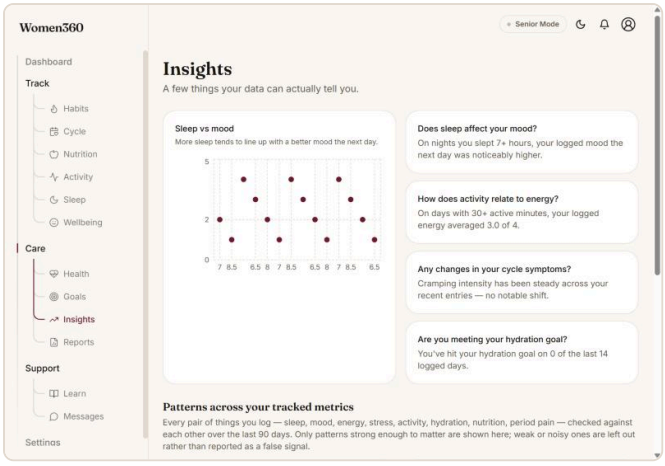


Figure 13. Insights.

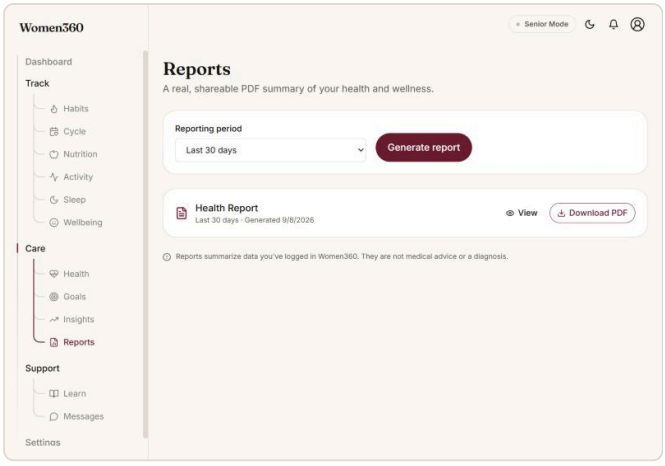


Figure 14. Reports.

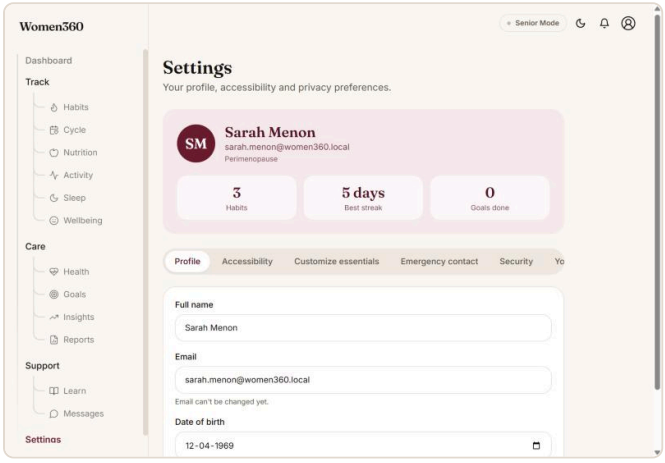


Figure 15. Settings and profile.

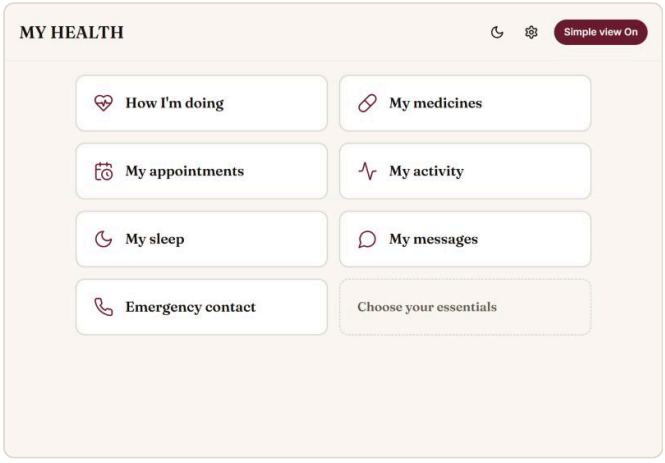


Figure 16. Senior Mode: large essentials instead of the full dashboard.

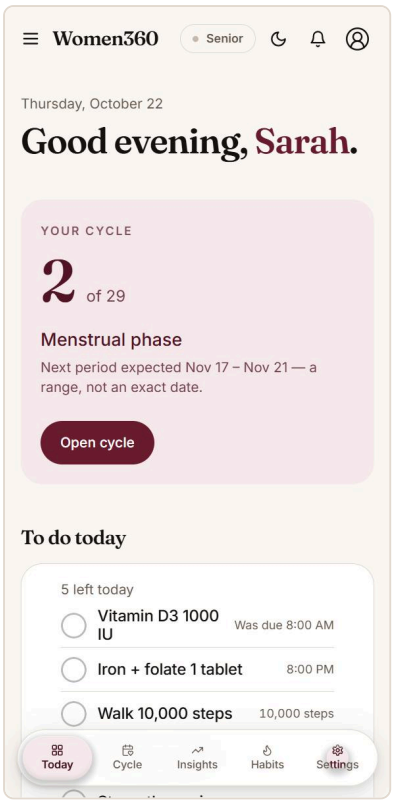


Figure 17. Phone layout (390 px wide) with the bottom navigation bar.

Responsive and accessible by design

- Desktop shows a sidebar; phones show a bottom bar; Senior Mode swaps the structure completely.
- No horizontal scrolling at phone (375 to 390 px) and desktop widths (checked while building each page).
- Semantic HTML, labelled inputs, visible focus, keyboard-operable controls and screen-reader announcements for status messages.
- Animations respect the reduced-motion setting.
- Light and dark themes follow the device unless the person chooses one.

14. Implementation

Area	Technology
Front end	React 18, TypeScript, Vite 5, Tailwind CSS 3, React Router, Recharts, GSAP (scroll reveals), jsPDF (report PDF)
Back end	Node.js 20, Express 4, TypeScript, zod validation, Helmet, CORS, rate limiting, Argon2, JSON Web Tokens
Database	PostgreSQL (Supabase), Prisma 5 ORM and migrations
Testing and quality	Vitest, Supertest, ESLint, strict TypeScript, GitHub Actions
Cloud	Vercel, Render, Supabase, GitHub; Capacitor for the optional Android wrapper

API conventions

All endpoints live under `/api/v1`. Responses use one envelope: `{ "ok": true, "data": ... }` or `{ "ok": false, "error": { "code", "message" } }`. Validation failures return 422, missing or invalid tokens 401, ownership or role violations 403, unknown records 404, and rate-limited requests 429.

Module	Representative endpoints
Auth	`POST /auth/register`, `/auth/login`, `/auth/refresh`, `/auth/logout`, `/auth/delete-account`

Users	`GET/PATCH /users/me`, `/users/me/export`, `/users/me/health-profile`, `/users/me/emergency-contact`
Cycle	`GET /cycle/summary`, `PUT/DELETE /cycle/entries/:date`, `PUT /cycle/profile`
Nutrition	`GET/POST /nutrition/meals`, `POST /nutrition/hydration`, `GET /nutrition/recent-meals`
Activity	`GET /activity/summary`, `POST/PATCH/DELETE /activity/entries`, `GET/POST/DELETE /activity/quick-log`
Goals and habits	`GET/POST/PATCH/DELETE /goals`, `/habits`, `PUT/DELETE /habits/:id/logs/:date`
Screening	`GET /screening`, `POST /screening/:id/records`, `DELETE /screening/:id/records/:recordId`
Insights and reports	`GET /insights`, `/insights/cycle-delay`, `/insights/low-mood`, `POST /reports`
Coach and admin	`GET /coach/assigned-women`, `/coach/women/:id/summary`, `GET /admin/users`, `PATCH /admin/users/:id/role`, `GET /admin/audit-log`

Cloud-related components

- **Configuration:** database URL, JWT secrets, CORS origins and the optional two-factor key are environment variables on Render; the front end gets only the public API base URL from Vercel. No secret is in the repository.
- **Migrations on deploy:** the Render build runs `prisma migrate deploy`, so a push that changes the schema also updates the cloud database.
- **Health check:** `GET /health` is used by Render and by the deployment test.
- **Database-backed lockout:** sign-in attempt counters live in PostgreSQL because the free-tier API sleeps and an in-memory counter would reset.
- **Security headers:** the Content-Security-Policy restricts scripts to the site itself and connections to the site and the API.

15. Testing

The server has 273 tests in 28 files and the front end 32 tests in 4 files, all passing when run on 22 October 2026. Server tests are integration tests: Supertest sends real HTTP requests to the application backed by a real PostgreSQL test database (not mocks), so authentication, validation, ownership checks and SQL are exercised together.

Area	Tests	What is covered
Authentication and security	73	Lockout and rate limits, password policy, token issue and expiry, sign out everywhere, two-factor (TOTP, backup codes), admin guards.
Roles and privacy	15	Coach assignment and consent, cross-user access refused (403), data export contains no credentials, account deletion requires the password.
Cycle and insights	56	Regular, irregular and perimenopausal cycles; thin-data handling; baseline-deviation detection; pregnancy prompt suppressed after menopause; correlations.
Tracking modules	88	Nutrition totals and decimals, sleep, activity and quick log, medicines and adherence, goals by kind, habits and streaks, backfilled days.
Preventive care	18	Age ranges, due and up-to-date logic, month-end dates, "not recorded" never reported as overdue, ownership.
Validation and misc.	23	Rejected malformed input, response envelope, app and database-URL guards.
Front end units	32	Sleep duration across midnight, 12/24-hour conversion, decimal checks, goal progress, quick-step sizing.

Selected test cases

#	Scenario	Expected	Result
T1	Register with a password that contains the person's name	Rejected with a validation error	**Pass**
T2	Five wrong sign-ins for one account, then the correct password	Account locked; the correct password is also refused until the lockout ends	**Pass**
T3	User B reads or deletes user A's cycle entries	Refused; A's records unchanged	**Pass**
T4	Call a protected route with no token	401	**Pass**
T5	Admin endpoint with the optional IP allow-list set, request from another address	Blocked before authentication is even attempted	**Pass**
T6	Highly variable cycle history	Low confidence and an irregularity note, shown as a range	**Pass**

T7	Log a screening twice for the same day	One record, no duplicates	**Pass**
T8	Export my data	Own data only, never credentials or another person's data	**Pass**
T9	Delete account with the wrong password	Refused; account intact	**Pass**
T10	Fix a missed habit day	That day marked done; today unchanged	**Pass**

Responsive, deployment and manual testing

- **Responsive:** pages were inspected at phone (375 to 390 px) and desktop widths for horizontal overflow and layout; the phone navigation and Senior Mode were checked separately.
- **Deployment (22 October 2026):** the live site returned HTTP 200; `GET /health` returned `{"ok":true,"data":{"status":"healthy"}}` (about 22 s on a cold start); newly added endpoints answered 401 without a token, which shows they are deployed and protected.
- **Not covered:** there is no automated browser end-to-end suite, and the front end has unit tests only for its pure functions. Accessibility was reviewed manually, not with an audit tool.

16. Deployment

1. **Database.** Create a Supabase project and copy its PostgreSQL connection string.
2. **API on Render.** Create a Blueprint from the repository. Render reads `render.yaml` (root directory `server`, Singapore region, free plan, health check `/health`). Set `DATABASE_URL`, `JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET` and `CORS_ORIGINS`. The build installs dependencies, generates the Prisma client, compiles TypeScript and runs `prisma migrate deploy`.
3. **Front end on Vercel.** Import the repository and set `VITE_API_BASE_URL` to the API's `/api/v1` URL. `vercel.json` adds the SPA rewrite and security headers.
4. **Connect.** Add the Vercel URL to `CORS_ORIGINS` on Render so the browser may call the API.
5. **Verify.** Open `/health`, register an account on the live site, and log a record.
6. **Updates.** A push to `main` deploys both services; schema changes migrate the database on the same deploy.

The step-by-step guide is `DEPLOYMENT.md` in the repository.

17. Results and Discussion

ID	Acceptance criterion	Status	Evidence
AC-01	A user can register and log in	**Met**	Live site; automated tests.
AC-02	Role-based access demonstrable for Woman, Coach and Admin	**Partly**	Enforced and tested on the API for all three roles; the web UI covers the Woman role only.
AC-03	Core records support CRUD	**Met**	Tracking modules, with tests.
AC-04	Dashboard with meaningful data and trends	**Met**	Figures 5 and 13.

AC-05	Reports from recorded data	**Met**	PDF export (Figure 14).
AC-06	REST APIs implemented and used	**Met**	About 110 endpoints used by the front end.
AC-07	Cloud database stores the data	**Met**	Supabase PostgreSQL.
AC-08	Responsive application	**Met**	Figures 5 and 17.
AC-09	Deployed to a public cloud URL	**Met**	https://women-360.vercel.app
AC-10	Source in a GitHub repository	**Met**	https://github.com/uthrahh/Women360

Observations

- The layered design let the front end be built against the real API from the start and let new modules (habits, screening, quick log) be added as additive migrations without touching existing data.
- Writing the guidance for screening as one reviewed file with named sources, and showing its status on screen, keeps medical claims out of the interface code.
- The free tier's cold start (about 22 s) is the main user-visible limit. It is a hosting-plan trade-off, not a design limit.
- The administrator and coach roles exist as protected APIs with tests but have no web screens; this is the main gap against SRS requirements FR-20, FR-21, UI-09 and UI-10.

18. Future Enhancements

- Step counts from a smartwatch: read steps from Android Health Connect (which Wear OS, Samsung and Fitbit watches sync into) through the Android app, and from Apple Health in an iOS app.
- Coach and admin web dashboards on top of the existing APIs, including the screen where a woman grants and revokes coach access.
- Clinical review of the screening schedule and educational content, then localisation by country (SRS §16).
- Push and email reminders; two-way coach messaging; contraception and pregnancy records.
- Wiring all onboarding steps to stored data; a browser end-to-end test suite; automated accessibility audits.
- A paid hosting tier (no cold start) and automated backups with a tested restore.

19. References

1. Women360 Software Requirements Specification (SRS), BCSE408L project documentation, 2026.
2. WHO. Women's health. who.int/health-topics/women-s-health
3. WHO. Menopause fact sheet. who.int/news-room/fact-sheets/detail/menopause
4. WHO. Cervical cancer fact sheet. who.int/news-room/fact-sheets/detail/cervical-cancer
5. WHO. Breast cancer fact sheet. who.int/news-room/fact-sheets/detail/breast-cancer
6. WHO. Guidelines on physical activity and sedentary behaviour (2020). who.int/publications/i/item/9789240015128

7. NCDC / Ministry of Health and Family Welfare, India. Training module for medical officers: prevention, control and population-level screening of hypertension, diabetes and common cancers (oral, breast, cervical), NPCDCS, 2017.
8. Documentation: React, Vite, Tailwind CSS, Express, Prisma, zod, Vitest, Supertest, Vercel, Render, Supabase, GitHub Actions.

20. Appendix

A. Run locally

`npm install` and `npm run dev` in the repository root for the front end; in `server/`: `npm install`, copy `.env.example` to `.env`, `npx prisma migrate deploy`, `npm run dev`. Tests: `npm test` in each folder. The demo seed refuses to run against a non-local database.

B. Environment variables (names only)

Server: `NODE_ENV`, `DATABASE_URL`, `JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET`, `JWT_ACCESS_TTL`, `JWT_REFRESH_TTL`, `CORS_ORIGINS`, `SUPPORT_EMAIL`, optional `TWO_FACTOR_ENCRYPTION_KEY`, `ADMIN_IP_ALLOWLIST`. Front end: `VITE_API_BASE_URL`.

C. Migrations

init · meal/sleep notes and goal progress · sleep quality scale and nutrition decimals · medication adherence · cycle context tags · auth hardening · two-factor · goal check-ins and feedback · health profile medicines · habits and goal kinds · screening records · activity quick log.

21. Project Access Details

> **Mandatory:** the application is accessible through the Internet. The credentials below are dedicated test accounts, not personal accounts. The first request after the service has been idle can take about 20 to 30 seconds while the free-tier API wakes up; please wait for the page to respond.

Live Application URL	https://women-360.vercel.app
Test User Username / Login	[TEST USER EMAIL - TO FILL]
Test User Password	[TEST USER PASSWORD - TO FILL]
Admin Username / Login	[ADMIN EMAIL - TO FILL, or write: not available in the web UI]
Admin Password	[ADMIN PASSWORD - TO FILL]

The administrator role is available through the protected `/api/v1/admin` endpoints; the web application does not include an administrator screen (see Section 17).

Final Submission Checklist

☐ Final Project Report ☐ Public Live URL ☐ Valid testing login credentials included ☐ GitHub Repository URL with source code ☐ PPT / Project Presentation ☐ Working application demonstration ☐ Both team members prepared to explain the implementation

Declaration

We confirm that the submitted project is our team's work, that the information and links provided are accurate, and that we can explain the implementation during evaluation. We understand the project guidelines regarding academic integrity and responsible use of AI tools.

Team Member 1: _____ Team Member 2: _____